MyCodeSchool

Exerciții — șiruri de caractere (2)

Python · proiectul `py-siruri` · Luca

Ce ai voie: tot din fișa 1 și din teorie · felii `text[1:]`, `text[-4:]` · `for litera in text`, `for cuvant in text.split()`, `for bucata in text.split(",")` · contor și acumulator de text · `.lower()`, `.upper()`, `.strip()`, `.replace()`, `.count()` · `int()`, `str()`, `"*" * 3`, f-string · `if` / `else`, `%`, `//`.
Încă NU folosim: liste construite de tine, `text.split()[0]`, `print` **în funcții**.
Textul gol `""` e cazul tău cu 0: unde apare în tabel, funcția trebuie să-l ducă la capăt fără eroare.

Cum lucrezi

Continui în același `siruri.py`, sub funcțiile din fișa 1. Fiecare funcție întoarce cu `return`; `main.py` afișează:

```
from siruri import ascunde_cardul


def main():
    print(ascunde_cardul("1234567812345678"))
    print(ascunde_cardul("123"))


if __name__ == "__main__":
    main()
```

Ieșire așteptată:

```
**********5678
123
```

La fiecare funcție nouă: o adaugi în `import` și pui câte un `print` pentru fiecare rând din tabelul ei.

La clasă

Prezentarea 1

```
def prezentare(nume: str, varsta: int) -> str:
```

Numele începe cu literă mare, restul literelor sunt mici.

apel	întoarce
`prezentare("luca", 15)`	`"Luca are 15 ani"`
`prezentare("ANA", 16)`	`"Ana are 16 ani"`
`prezentare("x", 9)`	`"X are 9 ani"`

Prima literă `.upper()` + felia de la poziția 1 `.lower()`. Textul final îl faci cu un f-string.

Cardul ascuns 2

```
def ascunde_cardul(numar_card: str) -> str:
```

Rămân vizibile doar ultimele 4 caractere; toate celelalte devin `*`.

apel	întoarce
`ascunde_cardul("1234567812345678")`	`"**********5678"`
`ascunde_cardul("98765")`	`"*8765"`
`ascunde_cardul("123")`	`"123"`
`ascunde_cardul("")`	`""`

O singură linie: `"*"` repetat de câte ori trebuie + o felie de la coadă. Încearcă în consolă `"*" * -1` înainte să scrii vreun `if`.

Cuvinte care încep cu o literă 3

```
def cuvinte_care_incep_cu(text: str, litera: str) -> int:
```

Nu contează dacă litera e mare sau mică, nici în text, nici în `litera`.

apel	întoarce
`cuvinte_care_incep_cu("Ana are alune", "a")`	`3`
`cuvinte_care_incep_cu("Luca invata Python", "P")`	`1`
`cuvinte_care_incep_cu("ana are mere", "x")`	`0`
`cuvinte_care_incep_cu("", "a")`	`0`

Contor + `for cuvant in ...split()`. Un cuvânt dat de `.split()` are mereu cel puțin o literă, deci `cuvant[0]` e sigur.

Mai grele

Numele, frumos scris 4

```
def formatare_nume(text: str) -> str:
```

Fiecare cuvânt cu literă mare la început și mici în rest, despărțite de **un singur** spațiu.

apel	întoarce
`formatare_nume("luca TODOR")`	`"Luca Todor"`
`formatare_nume("  ana   maria pop ")`	`"Ana Maria Pop"`
`formatare_nume("")`	`""`

Acumulator de text, ca la `initiale`, dar adaugi cuvântul întreg și un spațiu. Uită-te la ce iese la final: ai un spațiu în plus. Ce metodă din teorie îl scoate?

Mijlocul cuvântului 5

```
def litera_din_mijloc(text: str) -> str:
```

Lungime impară: litera din mijloc. Lungime pară: cele **două** litere din mijloc.

apel	întoarce
`litera_din_mijloc("abc")`	`"b"`
`litera_din_mijloc("Python")`	`"th"`
`litera_din_mijloc("a")`	`"a"`
`litera_din_mijloc("")`	`""`

`len(text) // 2` și `len(text) % 2`, ca la cifre. Scrie pe hârtie pozițiile din `"Python"` și vezi ce felie îți trebuie. Apoi întreabă-te: `""` are lungime pară sau impară, și pe ce ramură intră?

Suma din text 6

```
def suma_din_text(text: str) -> int:
```

Primești numere scrise cu virgulă între ele, ca într-un `input()`, și întorci suma lor.

apel	întoarce
`suma_din_text("3,10,7")`	`20`
`suma_din_text("5")`	`5`
`suma_din_text("")`	`0`

Acumulator + `for bucata in text.split(","):` + `int(...)`. Rulează rândul cu `""` primul și citește eroarea: ce bucăți îți dă `"".split(",")`?

Parola bună 7

```
def parola_buna(parola: str) -> bool:
```

O parolă e bună dacă are **cel puțin 8** caractere, cel puțin o cifră și cel puțin o literă mare.

apel	întoarce	apel	întoarce
`parola_buna("Python2026")`	`True`	`parola_buna("Pythonist")`	`False`
`parola_buna("Abcdef12")`	`True`	`parola_buna("Abc12")`	`False`
`parola_buna("python2026")`	`False`	`parola_buna("")`	`False`

Cifra: o variabilă `are_cifra = False` și o buclă cu `litera in "0123456789"`. Litera mare: o parolă are literă mare dacă nu e egală cu `parola.lower()`. `"Abcdef12"` are exact 8 caractere.

Provocări

Fără indicii.

Fără litere dublate 8

```
def fara_dubluri(text: str) -> str:
```

Dacă aceeași literă apare de mai multe ori **una după alta**, rămâne o singură dată.

apel	întoarce
`fara_dubluri("aaabccdd")`	`"abcd"`
`fara_dubluri("Mississippi")`	`"Misisipi"`
`fara_dubluri("abc")`	`"abc"`
`fara_dubluri("")`	`""`

Cea mai lungă serie 9

```
def cea_mai_lunga_serie(text: str) -> int:
```

Câte litere identice stau una după alta, cel mai mult.

apel	întoarce
`cea_mai_lunga_serie("aabbbbc")`	`4`
`cea_mai_lunga_serie("abc")`	`1`
`cea_mai_lunga_serie("abccc")`	`3`
`cea_mai_lunga_serie("")`	`0`

Anagrame 10

```
def este_anagrama(a: str, b: str) -> bool:
```

Două texte sunt anagrame dacă au exact aceleași litere, de același număr de ori, în altă ordine. Nu contează spațiile și nici literele mari.

apel	întoarce
`este_anagrama("Roma", "amor")`	`True`
`este_anagrama("Ana", "naa")`	`True`
`este_anagrama("ab", "abc")`	`False`
`este_anagrama("ana", "nan")`	`False`
`este_anagrama("", "")`	`True`

Înainte să faci push, treci fiecare funcție prin astea:

1. Am pus câte un `print` pentru **fiecare rând** din tabel, inclusiv cel cu `""`? Ieșirea e identică cu coloana „întoarce”?
2. Apare `None` undeva? Atunci o funcție afișează în loc să întoarcă, sau o ramură nu are `return`.
3. Am apelat `.lower()`, `.strip()` sau `.replace()` fără să păstrez rezultatul?
4. Fiecare `if` are o ramură pentru toate cazurile din tabel? Le-am urmărit pe hârtie pe cel mai scurt text?